INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA DA

PARAÍBA CAMPUS JOÃO PESSOA

CURSO SUPERIOR EM ENGENHARIA ELÉTRICA
INTRODUÇÃO À ENGENHARIA ELÉTRICA

Vitor Bruno Guilherme Tenorio Farias

PROTÓTIPO DE UMA ESTAÇÃO METEREOLÓGICA

JOÃO PESSOA – PB
2022

RESUMO

Este relatório tem o objetivo de demonstrar um protótipo de uma mini estação metereológica em tempo real. A ideia foi criar um sistema simples que possa demonstrar dados meteorológicos sobre o ambiente que o individuo está situado. Foi utilizado sensores simples que possa mostrar esses dados, com auxilio de um Arduino. Com o advento do hardware livre e sensores de baixo custo, torna-se viável a construção de estações meteorológicas precisas e acessíveis. Este projeto descreve a implementação de uma estação capaz de medir temperatura, umidade, pressão atmosférica e altitude, permitindo a coleta de dados ambientais de forma automatizada.

SUMÁRIO

INTRODUÇÃO..... 2

OBJETIVOS..... 2

FUNDAMENTAÇÃO TEÓRICA 2

Sobre a Metereologia e Prototipagem Arduino .....

Funcionamento dos Sensores .....

COMPONENTES UTILIZADOS 3
PROCEDIMENTO PRÁTICO..... 4
CONCLUSÃO 16
REFERÊNCIAS..... 17

INTRODUÇÃO

O monitoramento das variáveis atmosféricas desempenha um papel crucial na compreensão dos fenômenos climáticos e na tomada de decisões em diversos setores da sociedade. Desde o planejamento agrícola e a gestão de recursos hídricos até a prevenção de desastres naturais e o estudo do aquecimento global, a coleta de dados precisos sobre o estado da atmosfera é indispensável. Tradicionalmente, o acesso a esses dados dependia de estações meteorológicas profissionais de alto custo, muitas vezes distantes dos centros urbanos ou de propriedades rurais específicas.

Com o avanço da eletrônica de baixo custo e a popularização de plataformas de hardware livre, como o Arduino, tornou-se possível o desenvolvimento de sistemas de monitoramento ambiental personalizados e acessíveis. Uma estação meteorológica automatizada permite a coleta, o processamento e a exibição de grandezas fundamentais como temperatura, umidade relativa do ar e pressão atmosférica em tempo real, fornecendo informações valiosas sobre o microclima de uma determinada região.

Neste contexto, este projeto descreve o desenvolvimento de um protótipo de estação meteorológica simples. A importância deste trabalho reside na integração de sensores digitais e analógicos para a obtenção de parâmetros ambientais simples com fidelidade, demonstrando a viabilidade de sistemas de baixo custo para o suporte à pesquisa e ao monitoramento doméstico ou educacional. Além da relevância técnica, o projeto busca promover a sustentabilidade e a democratização da informação científica por meio da tecnologia.

OBJETIVOS

O objetivo principal deste projeto é desenvolver e implementar um protótipo de estação meteorológica automatizada, utilizando a plataforma de prototipagem eletrônica Arduino, capaz de monitorar e exibir em tempo real as variações de grandezas climáticas fundamentais.

FUNDAMENTAÇÃO TEÓRICA

Sobre a Meteorologia e Prototipagem Arduino

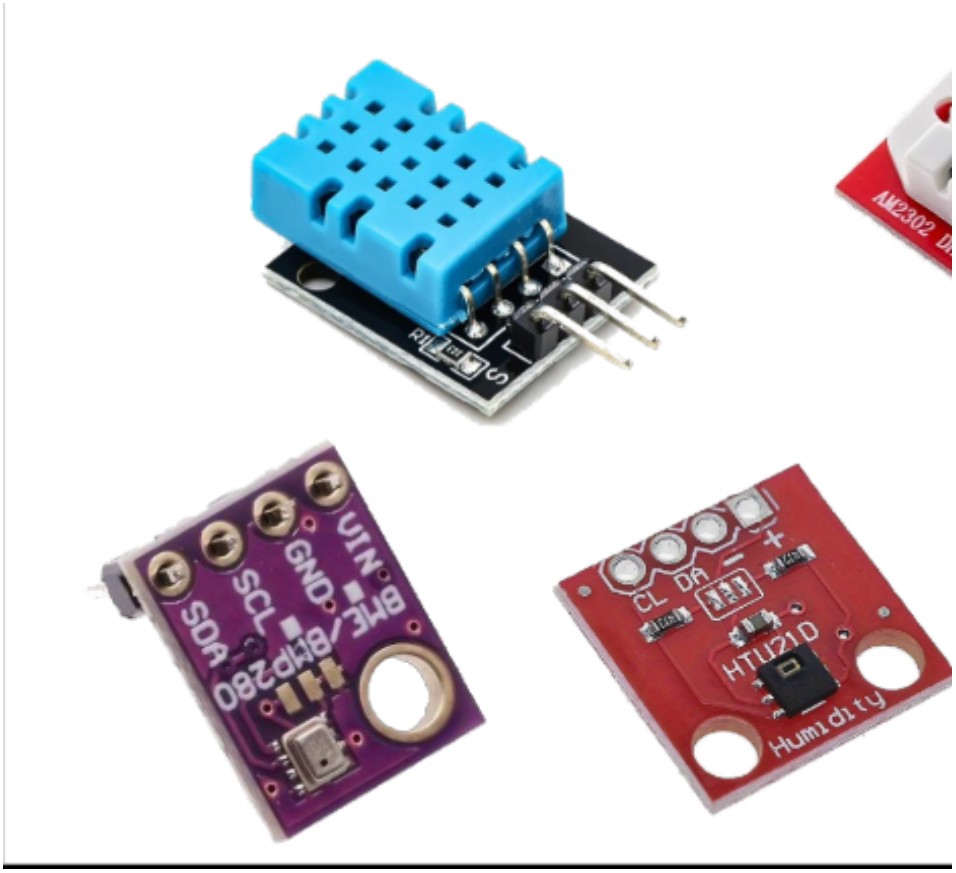
A meteorologia é a ciência que investiga as propriedades físicas da atmosfera terrestre e as interações que determinam o estado do tempo. O monitoramento contínuo de variáveis específicas é o que permite a geração de modelos preditivos e a análise de microclimas. No âmbito deste projeto, as grandezas fundamentais observadas são a temperatura do ar, que representa o nível de agitação térmica das moléculas; a humidade relativa, que indica a quantidade de vapor de água presente no

ar; e a pressão atmosférica, que é a força exercida pelo peso da coluna de ar. A variação desta última é um indicador fundamental para previsões de curto prazo, onde quedas bruscas de pressão geralmente sinalizam a aproximação de frentes de instabilidade.

O Arduino é uma plataforma de código aberto baseada em hardware e software de fácil utilização. No contexto de uma estação meteorológica, ele atua como a unidade de processamento central (CPU), sendo responsável por ler os sinais elétricos dos sensores, processar esses dados através de bibliotecas específicas e convertê-los em informações compreensíveis. A sua arquitetura permite a comunicação com diversos periféricos simultaneamente, garantindo a precisão no tempo de amostragem das leituras.

Funcionamento dos sensores

Para a aquisição de dados, o projeto utiliza sensores que oferecem alta precisão e baixo consumo de energia. Sensores como o DHT11 utilizam componentes capacitivos para medir a umidade e termístores para a temperatura. Já os sensores barométricos, baseados em tecnologia piezo-resistiva, medem a pressão absoluta com grande sensibilidade. Estes dispositivos são essenciais para transformar fenómenos físicos em dados digitais que podem ser armazenados ou visualizados.



Além dos sensores principais listados acima, há também um display que será utilizado para mostrar a exibição desses dados

COMPONENTES UTILIZADOS

Arduíno Uno
Cabo USB
3 protoboard mini 170 furos
Jumpers macho
3 Jumpers macho-femea

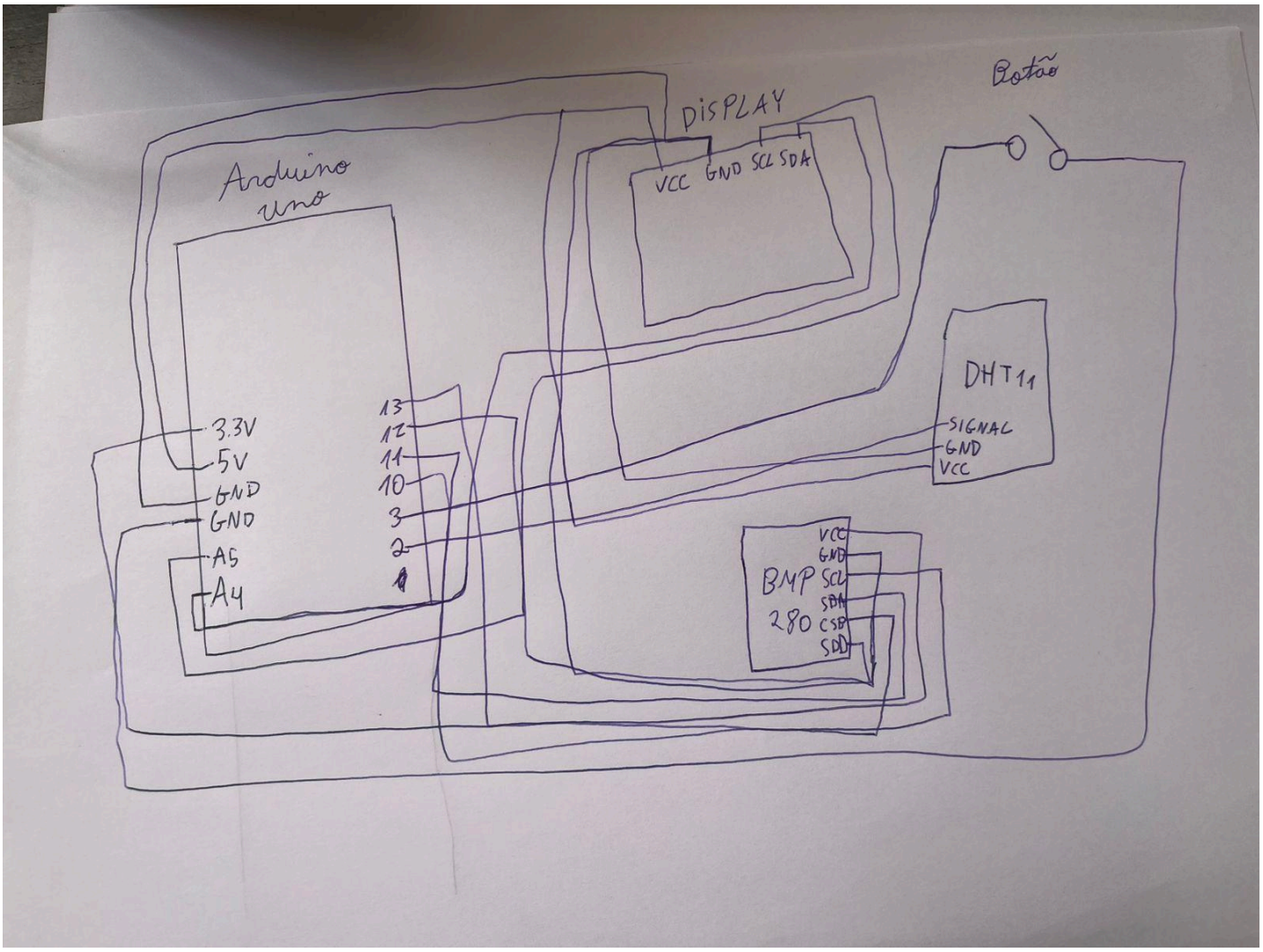
DHT11 Módulo de Temperatura e Umidade
BMP280 Módulo de Pressão Atmosférica, Altitude e Temperatura
1 Botão
1 Display 128x64 0.96 pol.

PROCEDIMENTO PRÁTICO

A montagem do protótipo foi dividida em duas etapas: hardware e software. Na parte de hardware, o sensor DHT11 foi conectado a um pino digital 2 do Arduino. O sensor BMP280 foi configurado via protocolo SPI, utilizando os pinos de comunicação serial síncrona para garantir a precisão na troca de dados, que são SCL, SDA, CSB e SDD, mas que no codigo são interpretados de outra maneira. O display LCD foi integrado via barramento I2C para facilitar a organização visual do circuito, além de que todos compartilham o mesmo Terra, sendo apenas o botão que utiliza um terramento diferente. Ambos Display e o DHT11 recebem 5 Volts enquanto o BMP280 recebe 3.3 Volts.

Contudo, devido algumas limitações em encontrar softwares que possam demonstrar a esquematica do circuito, foi feito o esquema a mão.

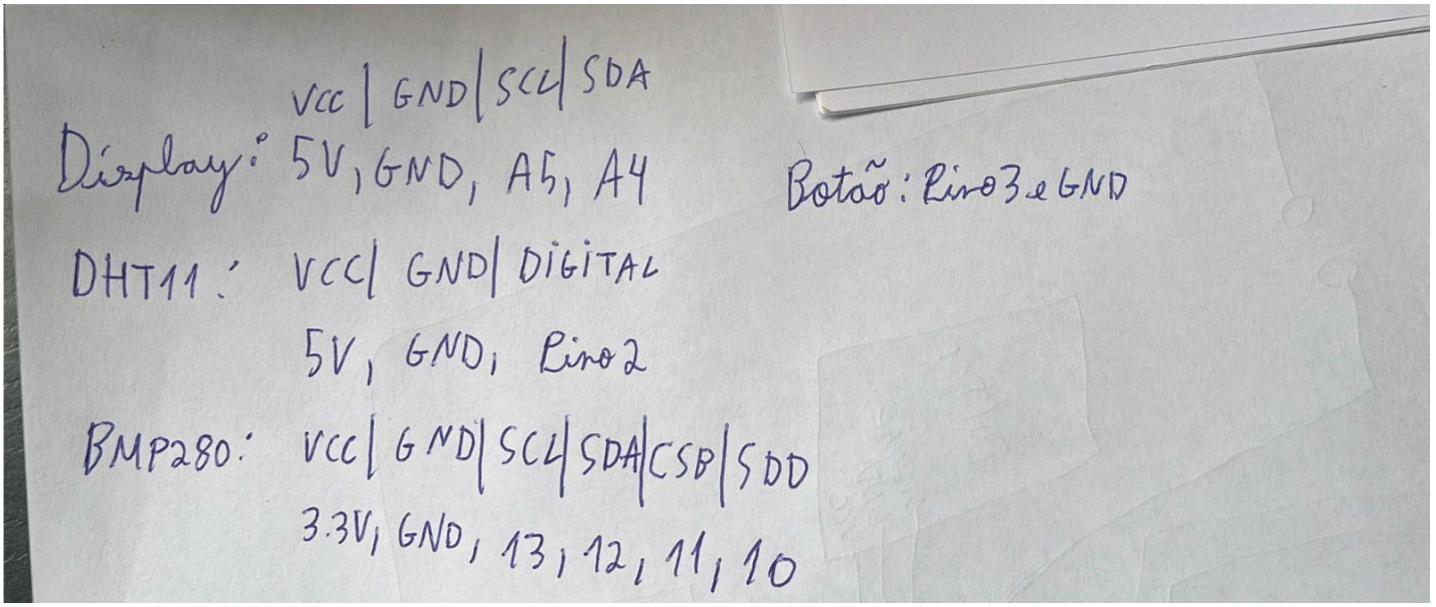
Esquema do projeto:



No Tinkercad e em vários softwares gratuitos não há simbolos para demonstrar os demais componentes.

Para facilitação da leitura a baixo descreve todos os pinos que os componentes estão conectados.

.



Abaixo segue a descrição do algoritmo e a codificação utilizada:

CÓDIGO

O programa começa incluindo certas bibliotecas, que são necessárias para conseguir fazer os sensores mostrarem os dados:

```
/**
 * Estacao Metereologica
 * Autor: Vitor Bruno
 */

#include <SPI.h>
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include <DHT11.h>
#include <Adafruit_BMP280.h>
```

Figura 1: Bibliotecas necessárias

- O Spi é um protocolo de comunicação serial sincrona de curto alcance, que utilizaremos para a comunicação com o BMP280.
- O Wire é a biblioteca nativa do Arduino que é focada no protocolo I2C, que permite o Arduino enviar e receber dados entre os sensores.
- O Adafruit_GFX é a biblioteca que contém todas as funções gráficas para utilizarmos no Display.

- O Adafruit_SSD1306 é a biblioteca necessária para conseguir se comunicar com o display.
- O DHT11 e Adafruit_BMP280 são as bibliotecas necessárias para conseguir se comunicar com o DHT11 e o BMP280.

```
#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
#define SCREEN_ADRESS 0x3C
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, -1);

DHT11 dht11(2);


#define BMP_SCK  13
#define BMP_MISO 12
#define BMP_MOSI 11
#define BMP_CS   10
Adafruit_BMP280 bmp(BMP_CS, BMP_MOSI, BMP_MISO, BMP_SCK);
```

Figura 2: Configurações necessárias pros Sensores e pro Display

A seguir temos que inicializar os nossos sensores e nosso display, para isso define-se a largura e altura do display e a inicialização padrão. Para o DHT11 apenas inclui o DHT11 dht11 (2), aonde o 2 representa a porta digital conectada ao arduino. Para o BMP280 define-se as portas SCK, MISO, MOSI e CS nas portas digitais 13, 12, 11 e 10, que são nossos SCL, SDA, CSB e SDD, para que possa ter a conexão SPI.

```
const int pinoBotao = 3;
int pagina = 1;
bool ultimoEstadoBotao = HIGH;

bool bmpOnline = false;
```

Figura 3: Configuração do Botão e uma Varíavel de Controle

Além de que definimos a lógica do botão, uma página para navegação no display e definimos uma variável de controle, ela serve para evitar que o Arduino trave ou fique tentando ler um sensor que não está lá.

```
void setup() {

  Serial.begin(9600);
  pinMode(pinoBotao, INPUT_PULLUP);

  Wire.begin();
  display.begin(SSD1306_SWITCHCAPVCC, SCREEN_ADRESS);

  // Tenta inicializar o BMP280 até 5 vezes
  for(int i = 0; i < 5; i++) {
    if (bmp.begin()) {
      bmpOnline = true;
      break;
    }
    delay(200);
  }

  display.clearDisplay();
  display.setTextColor(WHITE);
```

Figura 3: Começo do Setup

O setup funciona como a "lista de tarefas" que o Arduino executa uma única vez assim que liga. Ele começa preparando a comunicação serial e o barramento I2C, definindo a velocidade de conversa com os sensores. Em seguida, ele tenta "acordar" o display e cada sensor individualmente. Se o sensor responde, o código marca aquela variável (como o bmpOnline) como verdadeira para liberar o uso no resto do programa, e se não responde, ele apenas registra o erro e segue para o próximo, garantindo que o Arduino não trave se um fio estiver solto. Por fim, ele limpa a tela e mostra a sua mensagem inicial (Splash Screen) para confirmar que o sistema está pronto.

```
  display.clearDisplay();
  display.setTextColor(WHITE);

  // Splash Screen
  display.setTextSize(2);
  display.setCursor(22, 10);
  display.println("Estacao");
  display.setTextSize(1);
  display.setCursor(23, 35);
  display.println("Metereologica");
  display.display();
  delay(2000);

  display.clearDisplay();
  display.setCursor(20, 25);
  display.println("Por Vitor Bruno");
  display.display();
  delay(2000);
}
```

Figura 4: Configuração do Splash Screen

```
void loop() {
  bool estadoBotaoAtual = digitalRead(pinoBotao);
  if (ultimoEstadoBotao == HIGH && estadoBotaoAtual == LOW) {
    pagina++;
    if (pagina > 2) pagina = 1;
    delay(150);
  }
  ultimoEstadoBotao = estadoBotaoAtual;

  int temperatura, umidade;
  dht11.readTemperatureHumidity(temperatura, umidade);
```

Figura 5: Configuração do Loop

O começo do loop agora foca em duas partes: a vigilância do botão e a coleta do ar. Ele monitora o pino do botão constantemente para detectar quando você pressiona; se o sinal muda, ele atualiza a variável da página para que o display saiba o que mostrar em seguida. Logo depois, ele faz a leitura obrigatória do DHT11, que é o sensor principal para temperatura e umidade. Como o DHT11 não depende de endereços complexos como os outros, essa leitura é feita de forma direta e independente logo no início de cada repetição.

```
display.clearDisplay();
display.setCursor(0, 55);
display.print("Pagina: "); display.print(pagina);

if (pagina == 1) {
  display.setCursor(0, 0);
  display.println("--- DADOS DO AR ---");
  display.setTextSize(2);
  display.setCursor(0, 20);
  display.print(temperatura); display.println(" C");
  display.print(umidade);  display.println(" %");
  display.setTextSize(1);
}
else {
  display.setCursor(0, 0);
  display.println("--- ATMOSFERA ---");
  display.setCursor(0, 20);
  if(bmpOnline) {
    float pressao = bmp.readPressure() / 100.0F;
    float altitude = bmp.readAltitude(1013.25);
    display.print("Pressao: "); display.print(pressao, 1); display.println(" hPa");
    display.print("Altitude: "); display.print(altitude, 1); display.println(" m");
  } else {
    display.println("Sensor BMP OFFLINE");
    display.println("Verifique fios 10-13");
  }
}
```

Figura 6: Configuração do sistema de páginas e exibição dos dados no display

O fim do loop é a parte responsável por dar vida ao display, transformando os números captados pelos sensores em informações visuais organizadas por páginas.

Primeiro, ele limpa a tela usando a função display.clearDisplay(); para remover dados antigos e desenha o número da página atual no rodapé para você saber onde está navegando. Se a página selecionada for a número 1, o código foca nos dados do ar, exibindo a temperatura e a umidade vindas do DHT11 em um tamanho maior para facilitar a leitura.

Quando segura o botão ele mudará para a página 2, aonde demonstra os dados para a atmosfera: o sistema verifica se o BMP280 está respondendo e calcula a pressão e a altitude em tempo real. Se o sensor estiver desconectado, em vez de mostrar números errados, o código exibe um aviso de erro e sugere que você verifique a fiação, garantindo que sempre possa saber exatamente o que está acontecendo com o hardware.

```
display.display();
delay(100);
```

Figura 7: Finalização do código

Essa função final envia todos os desenhos e textos processados durante o loop para os pixels físicos da tela, pois sem esse comando o display permaneceria estático mesmo com os dados atualizados na memória. Logo após, é aplicado um pequeno intervalo de tempo para evitar que o processador trabalhe em uma velocidade excessiva, o que previne tremores visuais na imagem e garante que o sensor tenha tempo suficiente entre uma leitura e outra. Esse passo final fecha o ciclo de repetição, preparando o Arduino para reiniciar todo o processo de monitoramento instantaneamente.

Catalogo das constantes e variáveis:

```
#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
#define SCREEN_ADRESS 0x3C
#define BMP_SCK 13
#define BMP_MISO 12
#define BMP_MOSI 11
#define BMP_CS 10
const int pinoBotao = 3;
int pagina = 1;
bool ultimoEstadoBotao = HIGH;
bool bmpOnline = false;
bool estadoBotaoAtual;
```

int temperatura;
int umidade;
float pressao;
float altitude;
int i = 0;

Catalogo das funções criadas:

bmp.begin()
display.begin()
digitalRead()
dht11.readTemperatureHumidity()
bmp.readPressure()
bmp.readAltitude()
display.clearDisplay()
display.display()
display.setCursor()

Código completo:

```
/**
 * Estacao Metereologica
 * Autor: Vitor Bruno
 */

#include <SPI.h>
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include <DHT11.h>
#include <Adafruit_BMP280.h>

#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
#define SCREEN_ADRESS 0x3C
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, -1);

DHT11 dht11(2);


#define BMP_SCK  13
#define BMP_MISO 12
#define BMP_MOSI 11
#define BMP_CS   10
Adafruit_BMP280 bmp(BMP_CS, BMP_MOSI, BMP_MISO, BMP_SCK);


const int pinoBotao = 3;
int pagina = 1;
bool ultimoEstadoBotao = HIGH;

bool bmpOnline = false;
```

```
void setup() {

  Serial.begin(9600);
  pinMode(pinoBotao, INPUT_PULLUP);

  Wire.begin();
  display.begin(SSD1306_SWITCHCAPVCC, SCREEN_ADRESS);

  // Tenta inicializar o BMP280 até 5 vezes
  for(int i = 0; i < 5; i++) {
    if (bmp.begin()) {
      bmpOnline = true;
      break;
    }
    delay(200);
  }

  display.clearDisplay();
  display.setTextColor(WHITE);

  // Splash Screen
  display.setTextSize(2);
  display.setCursor(22, 10);
  display.println("Estacao");
  display.setTextSize(1);
  display.setCursor(23, 35);
  display.println("Metereologica");
  display.display();
  delay(2000);

  display.clearDisplay();
  display.setCursor(20, 25);
  display.println("Por Vitor Bruno");
  display.display();
  delay(2000);
}

void loop() {
  bool estadoBotaoAtual = digitalRead(pinoBotao);
  if (ultimoEstadoBotao == HIGH && estadoBotaoAtual == LOW) {
    pagina++;
    if (pagina > 2) pagina = 1;
    delay(150);
  }
  ultimoEstadoBotao = estadoBotaoAtual;

  int temperatura, umidade;
  dht11.readTemperatureHumidity(temperatura, umidade);

  display.clearDisplay();
  display.setCursor(0, 55);
  display.print("Pagina: "); display.print(pagina);
```

```
  if (pagina == 1) {
    display.setCursor(0, 0);
    display.println("--- DADOS DO AR ---");
    display.setTextSize(2);
    display.setCursor(0, 20);
    display.print(temperatura); display.println(" C");
    display.print(umidade);  display.println(" %");
    display.setTextSize(1);
  }
  else {
    display.setCursor(0, 0);
    display.println("--- ATMOSFERA ---");
    display.setCursor(0, 20);
    if(bmpOnline) {
      float pressao = bmp.readPressure() / 100.0F;
      float altitude = bmp.readAltitude(1013.25);
      display.print("Pressao: "); display.print(pressao, 1); display.println(" hPa");
      display.print("Altitude: "); display.print(altitude, 1); display.println(" m");
    } else {
      display.println("Sensor BMP OFFLINE");
      display.println("Verifique fios 10-13");
    }
  }

  display.display();
  delay(100);
}
```

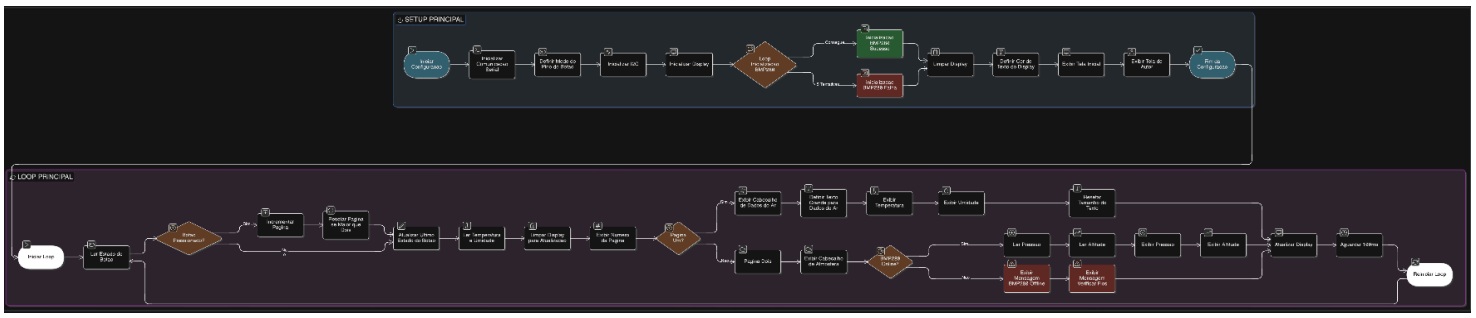


Figura 10 Diagrama do algoritmo

<u>Observação:</u> Link para melhor visualização do diagrama: https://app.eraser.io/workspace/QpejATkGKjFoEgnv2HTT?origin=share

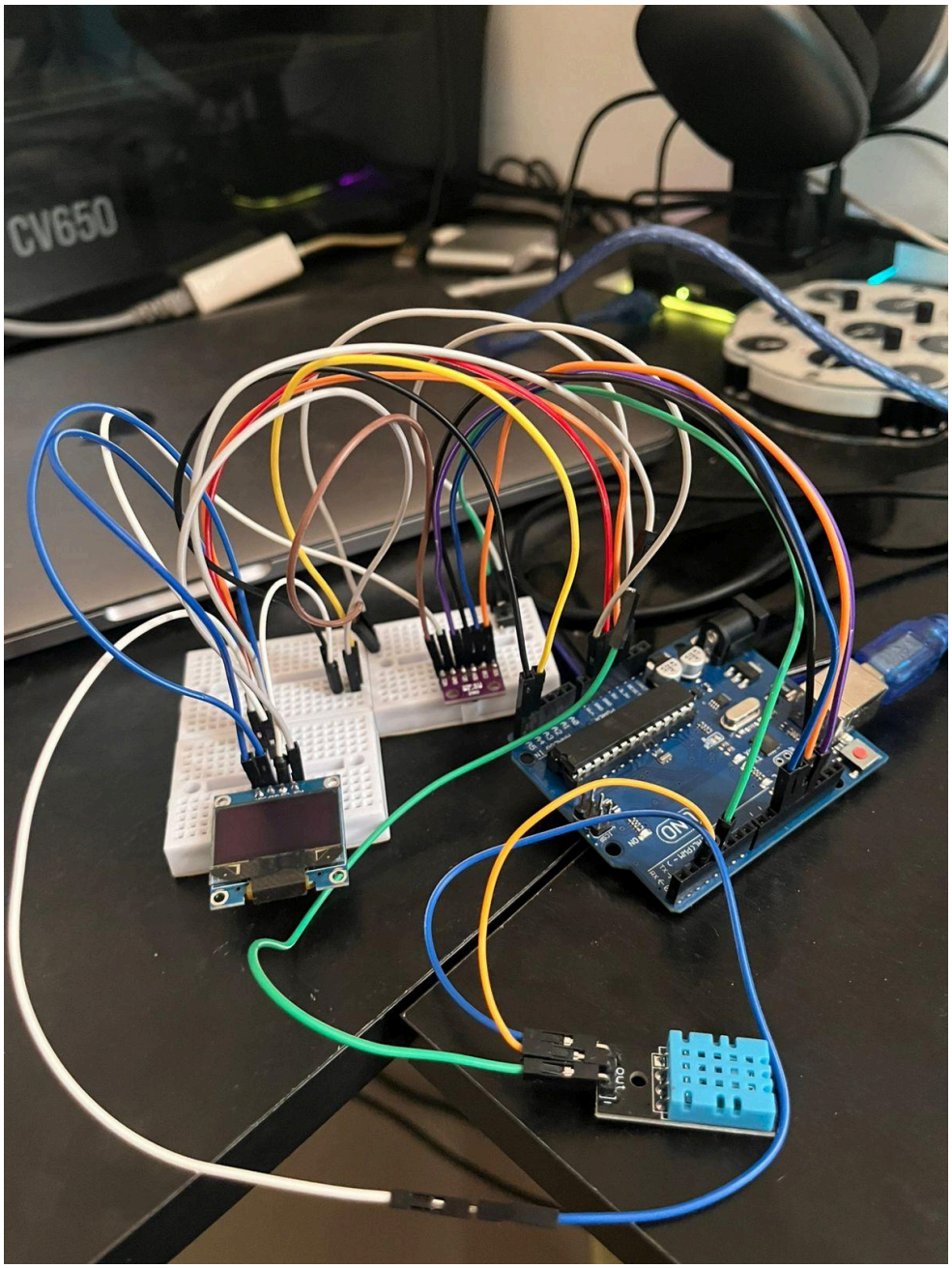


Figura 11: Visão dos componentes utilizados

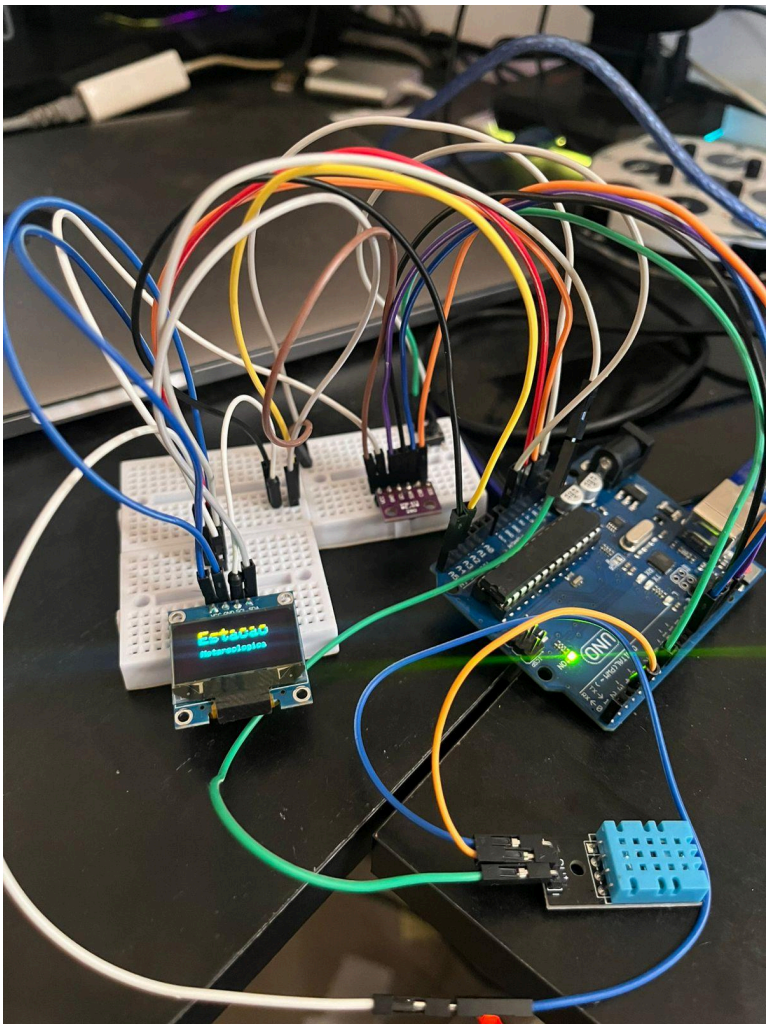


Figura 12: Inicialização da estação

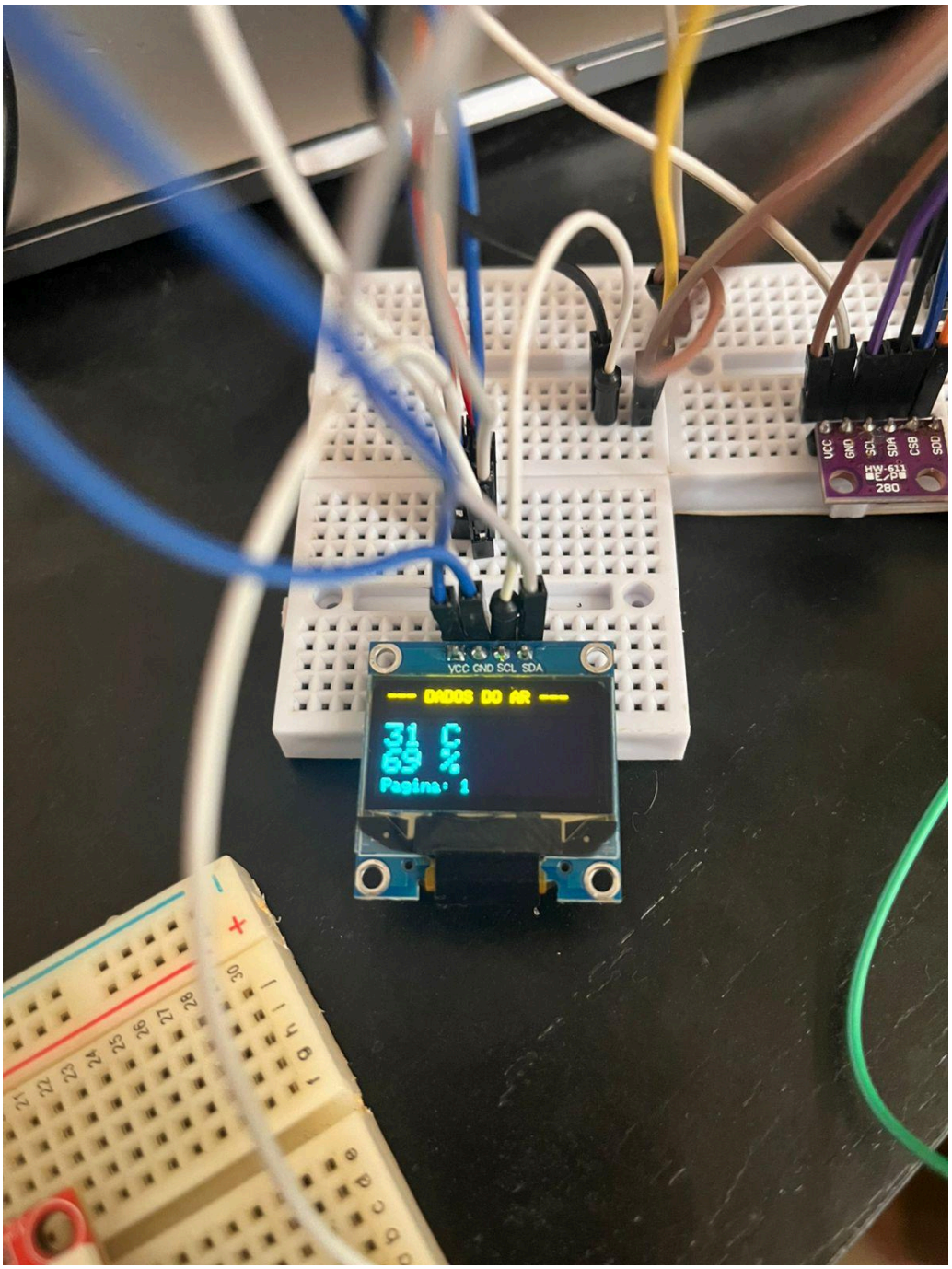


Figura 13: Sensor de umidade em funcionamento

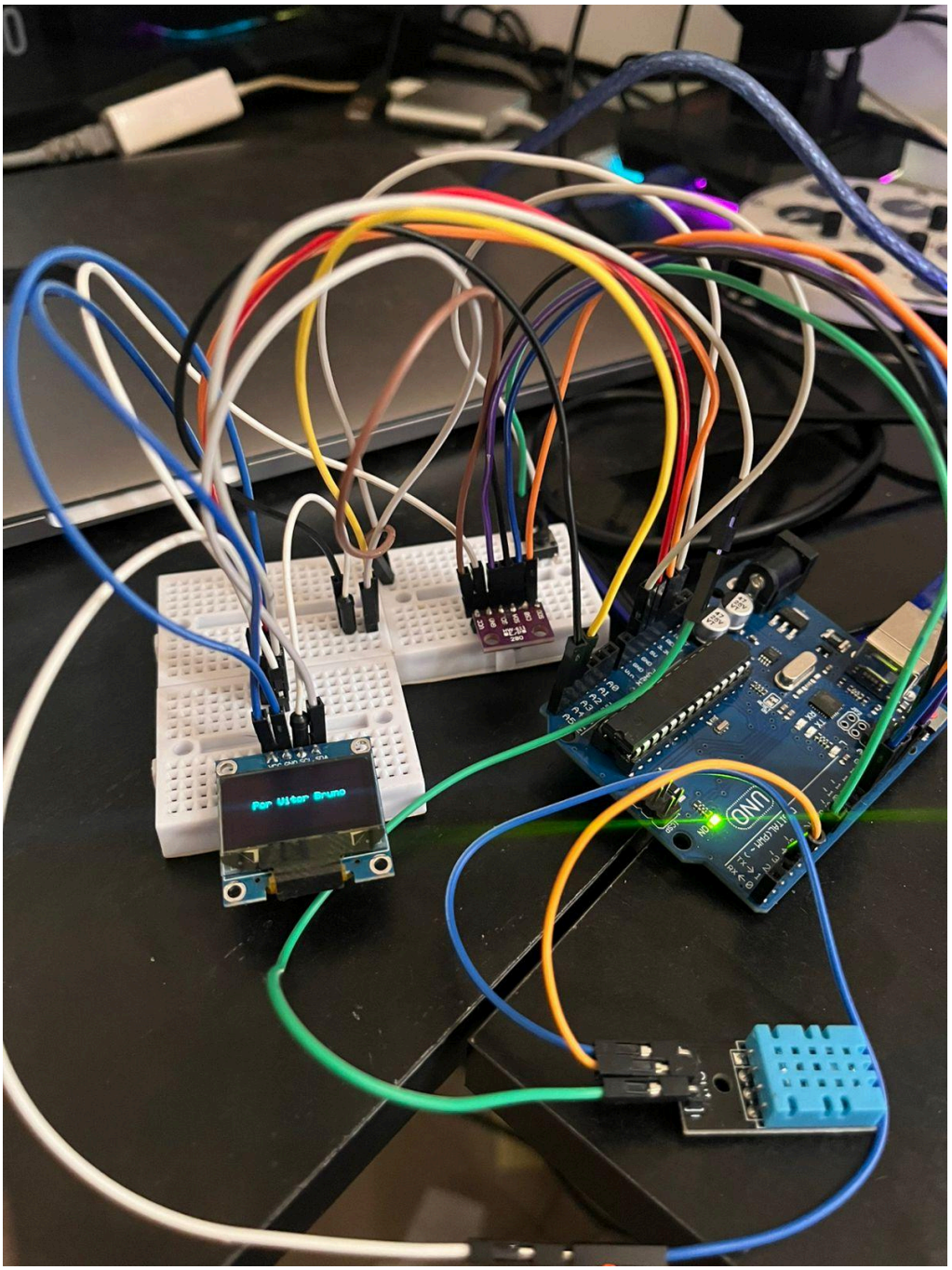


Figura 13: Projeto montado

CONCLUSÃO

O desenvolvimento dessa estação meteorológica automatizada tem os objetivos propostos solucionados, demonstrando ser uma solução viável e de baixo custo para o monitoramento climático. A integração do protocolo SPI com o sensor BMP280 permitiu leituras estáveis de pressão, enquanto o DHT11 forneceu dados satisfatórios de temperatura e umidade.

Conclui-se que o uso de plataformas de hardware livre como o Arduino facilita a democratização do acesso a dados meteorológicos. Como trabalhos futuros, uma implementação de um módulo Wi-Fi (como o ESP8266) poderá fazer o envio desses dados a uma plataforma de IoT, permitindo o acompanhamento remoto via internet.

7. REFERÊNCIAS

Sites:

- https://github.com/dhrubasaha08/DHT11?tab=readme-ov-file#examples
- https://github.com/adafruit/Adafruit_BMP280_Library/blob/master/examples/bmp280test/bmp280test.ino
- https://github.com/adafruit/Adafruit_SSD1306
- https://www.instructables.com/Simple-DIY-Weather-Station-With-DHT11-and-OLED-Dis/